
Exploring the CNN-LSTM Model Used for Predictive Cache Allocation and Testing Sparsity

Bryce Poonai
bwp7162@nyu.edu

1 Introduction

Modern storage systems are having higher demand on both storage cost and complexity. Allocating commonly used data in cache helps speed up these storage systems, but due to the increasing sophistication of these storage systems, effective caching becomes harder to do with traditional caching algorithms. Is there a way to improve this storage caching system using something else? According to the paper titled: “Adaptive Cache Management for Complex Storage Systems Using CNN-LSTM Based Spatiotemporal Prediction,” Deep learning models can be used to improve storage system cache allocation by training these models to adapt to data patterns within storage which would improve cache hits¹. They propose that this CNN-LSTM model performs the best by providing the lowest MSE and MAE out of all the other models when testing for cache allocation¹.

2 Problem Description

Using this model trained on real world storage data, how would it perform when we apply sparsity techniques to reduce model size, and which sparsity technique would make sense in this situation? How can we also improve the model to increase performance while maintaining its small footprint, so it can be used in portable situations.

3 Background/Related Work

3.1 Cache Demand Value

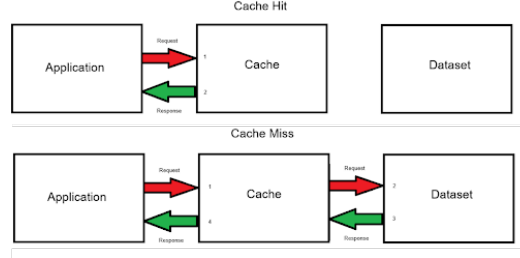
As mentioned in the introduction, the CNN-LSTM model improves cache allocation by adapting to storage patterns¹. In order to train this model, we would need to minimize its loss when compared to the cache demand value where y_t is the predicted cache demand value and y'_t is the actual cache demand value¹.

$$L = \frac{1}{N} \sum_{t=1}^T (y_t - y'_t)^2$$

The cache demand value is the amount of memory required to store frequently accessed data. It is determined by taking into consideration accumulated cache allocation size, cache hit rate, hot dataset sizes, access latency, dataset access frequency, etc¹. In this project, we will focus solely on the **accumulated cache allocation size** while taking into account the other aspects of the cache demand value since those other aspects require more involved data preprocessing.

3.2 Cache Hits

By improving cache allocation, we are theoretically increasing the amount of cache hits. A cache hit occurs when requested data is successfully retrieved within cache. Cache misses occur when requested data doesn't reside in cache which means it either has to go through an outer cache layer or the original dataset to be then copied into cache and then finally be used.



Increasing these cache hits lowers latency in data fetching thus increasing performance of the storage system.

4 Method

The objectives of this project is to build the testing platform for the datasets, model and sparsity testing; train the model using the cache traces dataset and optimize the model/hyperparameters to improve accuracy; obtain baseline metrics for model accuracy and model size; and test how the model performs and how much of a size benefit are we getting from using sparsity techniques on the model. Let's go into detail when it comes to building the project:

4.1 Cache Traces Dataset

There are multiple options for picking a cache traces dataset that is suitable for this project. Some examples include: Google Synthetic Traces, MetaStorage, and IBM Docker as potential candidates for this project. We instead use the MSR (Microsoft Research) Cambridge Traces which was also used in the Adaptive Cache Management paper and seems the simplest to work with compared to the other datasets. The dataset includes traces from multiple servers each with multiple volumes². We will use the src (source control) volumes for our project because of its mixture of read and write activity and repetition in certain data traces. For the training split we will use src1-0 and for our testing split we will use src1-1.

Here's what the cache traces file looks like:

Disknumber					
Timestamps	Hostname	R/W	Offset	Size	ResponseTime
128166373321652068	src1,0	Write	3173167104	36864	3126
128166373321653856	src1,0	Write	3154128896	4096	1337
128166373321653876	src1,0	Write	3154132992	4096	1318
128166373371652876	src1,0	Write	3744964608	4096	1998
128166373421651545	src1,0	Write	3173203968	32768	3009
128166373421653201	src1,0	Write	3154124800	4096	1353
128166373421653217	src1,0	Write	3154137088	4096	1336
128166373443526257	src1,0	Write	190584221696	4096	3156
128166373471650960	src1,0	Write	3173236736	36864	3273
128166373471652888	src1,0	Write	3154132992	4096	1345

Each cache trace has an associated timestamp for when the cache was issued. The disk and hostname depend on the file and remain the same for every trace. Cache operations are associated by either a read or a write. The offsets will be used as our identifiers for the caches. The sizes are recorded in bytes and will be used to calculate the accumulated cache allocation size. Each cache also had a recorded response time which shows the time taken for completion of the cache operation. These cache traces will be fed into the CNN-LSTM model in a feature matrix

4.2 Cache Partition (Ground Truths)

Now that we chose our dataset, we need to address how we're going to create our ground truth values to train the model. Each ground truth value will be the accumulated cache allocation size which is an approximation of the total size that should be allocated for the cache traces (we will call this the cache allocation value). However, the dataset is a continuous stream of cache traces, and it wouldn't be ideal to stream the entire cache traces set to predict one cache allocation value. Instead, let's partition the cache traces into separate groups to predict a value for each partition. This makes training more efficient and it can also be used to test how the model will perform when we scale this cache partition parameter.

When creating these ground truths with the partitions in mind, we also need to drop the duplicate values. We can use the cache offsets as labels for the caches to drop the repeating caches when it comes to the cache allocation value. This makes sense to do because if we have a cache operation that is a read and it's not in the partition, it should have an allocation. If we have a read and it's within cache, we don't need to allocate the same data in cache again. If we have an initial write to the cache partition, we should allocate for that. If we have a repeated write in the partition, we don't need to allocate for that cache since it already resides in the cache. We will choose 64 for the cache partition value and see how the model performs with this value in mind and with other values.

4.3 CNN-LSTM Model

Part of this project is exploring the CNN-LSTM model. There have been a couple changes made to make the model perform better without compromising on size, but there's always room for improvement. Here's what the architecture of CNN-LSTM model looks like:

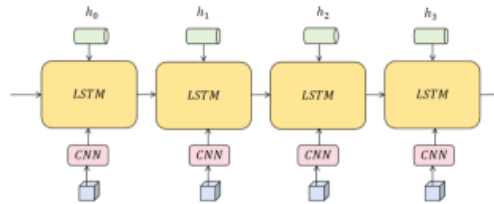


Figure 1 Overall architecture of the model

When implementing the project in code, the first thing you'll notice is we are matching the time steps with the cache partition size. This means we will have memory windows of each of the cache traces being fed into the model which will help optimize the model to correctly guess the right value for each partition. We are also feeding in a feature matrix, that contains all of the relevant columns from the cache traces dataset, into the model as our input. The CNN portion uses 1x1 convolutions since our features are being treated as 1x1 vectors. We are able to get spatial feature extraction between the features, but not within the features. We will cover how to improve this in the conclusion. The LSTM portion of the model has two LSTM layers. If we have two LSTM layers compared to one with both equating to the same size, the two layers seem to give better results. We can also better utilize the hidden states with two layers since we need to return the hidden states of the first LSTM layer which can be fed into the next layer which can allow for better pattern recognition.

We are also using mean squared error for our loss, mean absolute error to cross reference the paper, and mean absolute percent error to help calculate accuracy. It's also worth noting that the LSTM portion of the model is very sensitive to unscaled data and requires you to normalize your input data. Setting a range of 0.1 to 0.9 for our features, aside from the r/w column, seems to work as the range of 0 to 1 would cause divide by 0 errors when calculating the mean absolute percent error.

4.4 Sparsity Technique

With all of this building required for the project, there wasn't enough time to test multiple sparsity techniques and see which one would be the best. Instead, let's choose a sparsity technique that would

be best suited for this project. For this case, iterative pruning makes the most sense since we can fine tune the model with it after training. We will use a callback function and generate masks for each of the layers in the CNN-LSTM model. We will also choose local pruning since it's easier to implement than global pruning for this case.

5 Experiment Results

With the testing platform built, a baseline needs to be established so we can draw comparisons between the baseline and experiments.

5.1 Baseline

Using these parameters for our baseline:

Training Split: 100
Testing Split: 20
CP: 64
Epoch: 25
Batch Size 4

We are able to achieve an accuracy would hover around the 93-95% range with the model having a total size of around 1.2MB.

Here are what the predicted outputs look like compared to the ground truths:

Model Predictions	Ground Truths
[0.3256898]	[0.31428571]
[0.31904706]	[0.31507937]
[0.24606603]	[0.25]
[0.24774706]	[0.24761905]
[0.19722882]	[0.21031746]
[0.2890068]	[0.27857143]
[0.18007398]	[0.18333333]
[0.26519698]	[0.25634921]
[0.22461328]	[0.22619048]
[0.3112763]	[0.30634921]
[0.25543028]	[0.24365079]
[0.31150842]	[0.31587302]
[0.291344]	[0.28184524]
[0.2930853]	[0.29603175]
[0.36502466]	[0.35873016]
[0.3479253]	[0.33095238]
[0.3260115]	[0.30079365]
[0.3102887]	[0.31031746]

These values are both being batched and normalized, but we can see that the model is closely predicting the cache allocation values across all these cases.

5.2 Finetune with Iterative Pruning

When we finetune the baseline model with these parameters, we are able to achieve this:

Epoch: 5
Sparsity Level: 4

```

Fine-tuning with ManualPruningCallback complete.
10/10 4s 17ms/step - loss: 0.0012 - mae: 0.0209
10/10 6s 360ms/step
Manual Pruned Test loss: 0.0012, Manual Pruned Test MAE: 0.0209, Manual Pruned Accuracy: 95.93%

```

```

Original Model Size: 1233.43 KB
Compressed Model Size: 1015.11 KB
Size Reduction: 17.70%

```

We can see that accuracy increases and we achieve a size reduction of around 17.7% with the model being at around 1MB when we compress it.

Now let's see what we can achieve when we scale sparsity. Using the same parameters with a baseline model, let's apply different levels of sparsity with the same finetuning iterative pruning parameters:

20% Sparsity:

```
Fine-tuning with ManualPruningCallback complete.
10/10 ----- 4s 17ms/step - loss: 7.3570e-04 - mae: 0.0118
10/10 ----- 5s 320ms/step
Manual Pruned Test Loss: 0.0007, Manual Pruned Test MAE: 0.0118, Manual Pruned Accuracy: 97.61%
```

```
*** Original Model Size: 1233.43 KB
Compressed Model Size: 1070.14 KB
Size Reduction: 13.24%
```

40% Sparsity:

```
Fine-tuning with ManualPruningCallback complete.
10/10 ----- 6s 17ms/step - loss: 0.0011 - mae: 0.0232
10/10 ----- 5s 335ms/step
Manual Pruned Test Loss: 0.0011, Manual Pruned Test MAE: 0.0232, Manual Pruned Accuracy: 95.02%
```

```
Original Model Size: 1233.43 KB
Compressed Model Size: 1029.62 KB
Size Reduction: 16.52%
```

60% Sparsity:

```
Fine-tuning with ManualPruningCallback complete.
10/10 ----- 4s 10ms/step - loss: 9.5200e-04 - mae: 0.0197
10/10 ----- 6s 371ms/step
Manual Pruned Test Loss: 0.0010, Manual Pruned Test MAE: 0.0197, Manual Pruned Accuracy: 96.10%
```

```
*** Original Model Size: 1233.43 KB
Compressed Model Size: 962.41 KB
Size Reduction: 21.97%
```

80% Sparsity:

```
Fine-tuning with ManualPruningCallback complete.
10/10 ----- 5s 10ms/step - loss: 4.5492e-04 - mae: 0.0116
10/10 ----- 5s 320ms/step
Manual Pruned Test Loss: 0.0005, Manual Pruned Test MAE: 0.0116, Manual Pruned Accuracy: 97.83%
```

```
Original Model Size: 1233.43 KB
Compressed Model Size: 899.41 KB
Size Reduction: 27.08%
```

90% Sparsity:

```
Fine-tuning with ManualPruningCallback complete.
10/10 ----- 5s 17ms/step - loss: 7.0001e-04 - mae: 0.0246
10/10 ----- 5s 327ms/step
Manual Pruned Test Loss: 0.0007, Manual Pruned Test MAE: 0.0246, Manual Pruned Accuracy: 94.42%
```

```
Original Model Size: 1233.43 KB
Compressed Model Size: 857.27 KB
Size Reduction: 30.50%
```

The size reductions seem to scale from 20-90% with 90% giving a size reduction of over 30%. Accuracy seems to also be maintained across all these sparsity levels. When testing 90% sparsity, accuracy does seem to drop.

When we run the same test with an epoch count of 10, the size reductions don't seem to improve and accuracy, across the board, is slightly worse compared to fine tuning with 5 epochs.

5.3 Scaling Cache Partition Size

The last bit of testing, that is more so about exploring the CNN-LSTM model, is how the cache partition affects performance. We have the results of training the model with CP=64, but what about values of 16, 256 and 1024?

CP = 16:

```
Model created successfully.
40/40 ----- 1s 7ms/step - loss: 2.7956e-04 - mae: 0.0153
40/40 ----- 2s 27ms/step
Test Loss: 0.00027956359554882155, Test MAE: 0.01525686495806844, Accuracy: 95.9893458895091
Model: "sequential_1"
```

CP = 256:

```
Model created successfully.
3/3 ----- 19s 78ms/step - loss: 0.0010 - mae: 0.0239
3/3 ----- 26s 7s/step
Test Loss: 0.001036095549352467, Test MAE: 0.023852156475186348, Accuracy: 93.9191979802154
Model: "sequential_2"
```

CP = 1024:

```
1/1 ----- 120s 120s/step - loss: 0.0236 - mae: 0.0889
WARNING:tensorflow:5 out of the last 44 calls to <function TensorFlowTrainer.make_predict_fu
1/1 ----- 47s 47s/step
Test Loss: 0.023557597771286964, Test MAE: 0.08890338987112045, Accuracy: 55.47519071960678
Model: "sequential_3"
```

The model seems to perform well when using cache partitions of 16 and 256. 1024 seems to take much longer for compiling the model and accuracy seems to suffer. This is due to the time scale value being linked to the cache partition, so a solution to this is to decouple the cache partition with the time scale and use a time scale value of around 200-300.

6 Conclusion

Firstly, what are the next steps to continue this project to optimize for the cache demand value with the use of the cache allocation value? The CNN portion of the model is taking in 1x1 inputs and using 1x1 convolutions for spatial feature extraction which works for between the features, but not within each feature. Instead of linearly feeding in cache traces values, like we did here, cache traces can be grouped into spatial maps labeled by their cache offsets. The model can then be fed snapshots of these spatial maps to process cache demand values of these snapshots. This will give the model a better understanding of which cache traces are more actively being used and should be more likely allocated for in the cache demand value. 3x3 convolutions can now be used to achieve spatial feature extraction within these snapshots. As for the LSTM portion of the model, temporal aspects of the data would need to be factored into the ground truth values. Although figuring out how the cache demand value changes when incorporating access frequency and cache lifecycle shifting is another whole different problem to tackle.

Next, what are the potential use cases for a project like this? This model can be deployed in storage systems with memory constrained memory controllers such as systems with embedded systems or IoT devices as possibilities. With this cache partition, data can be stored using parallel storage data controllers where data is stored within multiple storage nodes with the help of this model.

Finally, what did we achieve with this project? We created a cache partition scheme to split the cache traces dataset into segments that the model can use to train and interpret more efficiently in a linear method. We tested and optimized the CNN-LSTM model to better predict cache allocation values that can be incorporated into determining the cache demand value along with the other features from the cache traces dataset. Lastly, we saw that sparsity scales really well with the CNN-LSTM model, in this application, by decreasing the model size while maintaining accuracy when fine tuned with iterative pruning.

7 References

[1]Wang, X., Li, X., Wang, L., Ruan, T., & Li, P. (2024, November 19). Adaptive Cache Management for complex storage systems using CNN-LSTM-based spatiotemporal prediction. arXiv.org. <https://arxiv.org/abs/2411>.

[2]Narayanan, D., Donnelly, A., & Rowstron, A. (n.d.). Write off-loading: Practical power management for Enterprise storage. Usenix. https://www.usenix.org/legacy/event/fast08/tech/full_papers/narayanan/narayanan_html/index.html

8 Project Link

Here's the project shared in Google Colab if you guys want to take a look at the code:
<https://colab.research.google.com/drive/1gb9YYdxzYBn91wLfFX70nVauum9F9AV1?usp=sharing>